

```
import nltk
import numpy as np
import pandas as pd
```

```
from nltk.corpus import movie_reviews
#from utils import process_reviews, build_freqs
```

```
nltk.download('movie_reviews')
```

```
[nltk_data] Downloading package movie_reviews to /root/nltk_data...
[nltk_data] Package movie_reviews is already up-to-date!
True
```

```
pos_fileids = movie_reviews.fileids('pos')
neg_fileids = movie_reviews.fileids('neg')

all_positive_reviews = [movie_reviews.raw(fileid) for fileid in pos_fileids]
all_negative_reviews = [movie_reviews.raw(fileid) for fileid in neg_fileids]
```

```
len(all_positive_reviews)
```

```
1000
```

```
len(all_negative_reviews)
```

```
1000
```

```
test_pos = all_positive_reviews[800:]
train_pos = all_positive_reviews[:800]
test_neg = all_negative_reviews[800:]
train_neg = all_negative_reviews[:800]
```

```
train_x = train_pos + train_neg
test_x = test_pos + test_neg

train_y = np.append(np.ones(len(train_pos)), np.zeros(len(train_neg)))
test_y = np.append(np.ones(len(test_pos)), np.zeros(len(test_neg)))
```

```
len(train_x)
```

```
1600
```

```
train_x[0]
```

```
'films adapted from comic books have had plenty of success , whether they\'re about superheroes ( batman , superman , spawn ) , or geared toward kids ( casper ) or the arthouse crowd ( ghost world ) , but there\'s never really been a comic book li ke from hell before . \nfor starters , it was created by alan moore ( and eddie campbell ) , who brought the medium to a wh ole new level in the mid \'80s with a 12-part series called the watchmen . \nto say moore and campbell thoroughly researche d the subject of jack the ripper would be like saying michael jackson is starting to look a little odd . \nthe book ( or "
```

```
train_x[1599]
```

```
' * * * be warned . . . \nthe following review contains some harsh language * * * \nthe blair witch project . \nquite possi bly the least scariest movie of all time . \nif you want to see real terror on the big screen go back and watch any scene w ith jar jar binks in the phantom menace , because this movie is not scary or even remotely creepy . \nmy colleague and frie nd chuck dowling wrote that if less is more , then the blair witch filmmakers must have thought that nothing is more . \nan d this worked for him . \nwell , i\'m sorry , but nothing . . . \nis nothing ! ! ! ! \nnothing is not more . \nit\'s noth
```

```
import re
import string

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import TweetTokenizer
```

```
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
True
```

```
def process_review(review):
    """
```

Очищення тексту, токенизація, видалення стоп-слів та стемінг

"""

```
stemmer = PorterStemmer()
stopwords_english = stopwords.words('english')
review = re.sub(r'\\$\\w*', '', review)
review = re.sub(r'^RT[\\s]+', '', review)
review = re.sub(r'https?:/[\\^\\s\\n\\r]+', '', review)
review = re.sub(r'#', '', review)
tokenizer = TweetTokenizer(
    preserve_case=False, strip_handles=True, reduce_len=True
)
review_tokens = tokenizer.tokenize(review)

reviews_clean = []
for word in review_tokens:
    if (
        word not in stopwords_english
        and any(ch.isalpha() for ch in word)
    ):
        stem_word = stemmer.stem(word)
        reviews_clean.append(stem_word)

return reviews_clean
```

train_x[0]

'films adapted from comic books have had plenty of success , whether they\\'re about superheroes (batman , superman , spawn) , or geared toward kids (casper) or the arthouse crowd (ghost world) , but there\\'s never really been a comic book li ke from hell before . \\nfor starters , it was created by alan moore (and eddie campbell) , who brought the medium to a wh ole new level in the mid \\'80s with a 12-part series called the watchmen . \\nto say moore and campbell thoroughly researche d the subject of jack the ripper would be like saying michael jackson is starting to look a little odd . \\nthe book (or "

process_review(train_x[0])

```

',
'strong',
'violenc',
'gore',
'sexual',
'language',
'drug',
'content']

```

```
train_y
```

```
array([1., 1., 1., ..., 0., 0., 0.])
```

```
len(train_y)
```

```
1600
```

```

def build_freqs(reviews, ys):
    """
    Побудова словника частотностей для пар

    """
    yslst = np.squeeze(ys).astype(int).tolist()
    freqs = {}
    for y, review in zip(yslst, reviews):
        for word in process_review(review):
            pair = (word, y)
            freqs[pair] = freqs.get(pair, 0) + 1
    return freqs

```

```
freqs = build_freqs(train_x, train_y)
```

```
len(freqs)
```

```
43376
```

```

pos_top = sorted([(w, c) for (w, y), c in freqs.items() if y == 1], key=lambda t: -t[1])
neg_top = sorted([(w, c) for (w, y), c in freqs.items() if y == 0], key=lambda t: -t[1])

```

```
print("Слів у позитивному класі:", len(pos_top))
```

```
Слів у позитивному класі: 22283
```

```
print("Слів у негативному класі:", len(neg_top))
```

```
Слів у негативному класі: 21093
```

```
print("Позитивний клас, найчастіші:", pos_top[:15])
```

```
Позитивний клас, найчастіші: [('film', 4635), ('one', 2407), ('movi', 2377), ('like', 1550), ('character', 1546), ('make', 1250), ('
```

```
print("Позитивний клас, найрідші:", pos_top[-15:])
```

```
Позитивний клас, найрідші: [('perjur', 1), ('good-guy', 1), ('frown', 1), ('grizzli', 1), ('affair-a-week', 1), ('venora', 1), ('
```

```
print("Негативний клас, найчастіші:", neg_top[:15])
```

```
Негативний клас, найчастіші: [('film', 3806), ('movi', 2828), ('one', 2121), ('like', 1652), ('character', 1394), ('get', 1367), ('
```

```
print("Негативний клас, найрідші:", neg_top[-15:])
```

```
Негативний клас, найрідші: [('grouch', 1), ('unwork', 1), ('jailbird', 1), ('pantomim', 1), ('henpeck', 1), ('bink', 1), ('r
```

```

def sigmoid(z):
    h = 1 / (1 + np.exp(-z))
    return h

```

```
result = []
```

```

for z in np.arange(-10, 10, 0.1):
    result.append(sigmoid(z))

```

```
result
```

```

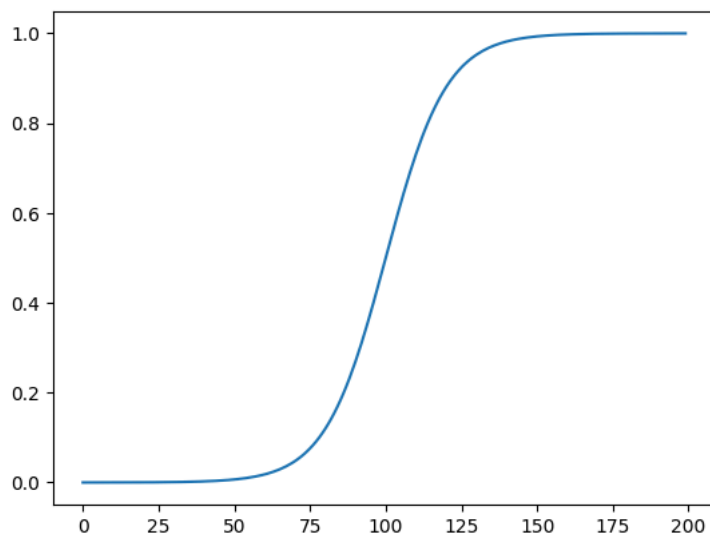
np.float64(0.9852259683067263),
np.float64(0.9866130821723345),
np.float64(0.9878715650157252),
np.float64(0.9890130573694063),
np.float64(0.9900481981330951),
np.float64(0.9909867013471517),
np.float64(0.9918374288468397),
np.float64(0.9926084586557177),
np.float64(0.9933071490757148),
np.float64(0.9939401985084155),
np.float64(0.9945137011005493),
np.float64(0.9950331983499427),
np.float64(0.9955037268390586),
np.float64(0.9959298622841037),
np.float64(0.9963157601005639),
np.float64(0.9966651926925865),
np.float64(0.9969815836752914),
np.float64(0.9972680392369888),
np.float64(0.9975273768433651),
np.float64(0.9977621514787236),
np.float64(0.9979746796109501),
np.float64(0.998167061057507),
np.float64(0.9983411989198255),
np.float64(0.998498817743263),
np.float64(0.9986414800495709),
np.float64(0.9987706013787224),
np.float64(0.9988874639671396),
np.float64(0.9989932291799142),
np.float64(0.9990889488055994),
np.float64(0.9991755753136017),
np.float64(0.9992539711661631),
np.float64(0.999324917269367),
np.float64(0.9993891206405656),
np.float64(0.9994472213630764),
np.float64(0.9994997988929205),
np.float64(0.9995473777767595),
np.float64(0.9995904328350139),
np.float64(0.9996293938593735),
np.float64(0.9996646498695334),
np.float64(0.9996965529699712),
np.float64(0.9997254218438986),
np.float64(0.9997515449181605),
np.float64(0.9997751832297667),
np.float64(0.9997965730219448),
np.float64(0.9998159280950366),
np.float64(0.9998334419352227),
np.float64(0.9998492896419403),
np.float64(0.9998636296729204),
np.float64(0.9998766054240137),
np.float64(0.9998883466593704),
np.float64(0.9998989708060922),
np.float64(0.9999085841261478),
np.float64(0.9999172827771484),
np.float64(0.9999251537724895),
np.float64(0.9999322758503801),
np.float64(0.9999387202603833),
np.float64(0.9999445514752772),
np.float64(0.9999498278353162)]

```

```
from matplotlib import pyplot as p
```

```
p.plot(result)
```

```
[<matplotlib.lines.Line2D at 0x7f28f1e38cd0>]
```



```
def gradientDescent(x, y, theta, alpha, num_iters):
    m = x.shape[0]
    y = y.reshape(-1, 1)
    history = []
    for i in range(num_iters):
        h = sigmoid(np.dot(x, theta))
        h_safe = np.clip(h, 1e-15, 1 - 1e-15)
        J = (-1/m) * np.sum(y * np.log(h_safe) + (1-y) * np.log(1 - h_safe))
        theta = theta - (alpha/m) * np.dot(x.T, (h - y))
        history.append(J)
    return J, theta, history
```

```
N_pos = sum(v for (w, y), v in freqs.items() if y == 1)
N_neg = sum(v for (w, y), v in freqs.items() if y == 0)
V = len({w for (w, y) in freqs})
```

```
def extract_features(review, freqs):
    words = process_review(review)
    x = np.zeros((1, 5))
    x[0, 0] = 1.0
    if not words:
        return x
    pos_s = neg_s = lo = strong = 0.0
    for w in words:
        p = freqs.get((w, 1), 0)
        n = freqs.get((w, 0), 0)
        pol = (p - n) / (p + n + 1)
        pos_s += max(pol, 0)
        neg_s += max(-pol, 0)
        lo += np.log((p + 1) / (N_pos + V)) - np.log((n + 1) / (N_neg + V))
        strong += (pol > 0.3) - (pol < -0.3)
    m = len(words)
    x[0, 1] = pos_s / m
    x[0, 2] = neg_s / m
    x[0, 3] = lo / m
    x[0, 4] = strong / m
    return x
```

```
X = np.vstack([extract_features(r, freqs) for r in train_x])
mu, sigma = X[:, 1:].mean(axis=0), X[:, 1:].std(axis=0)
X[:, 1:] = (X[:, 1:] - mu) / sigma
```

```
def predict_review(review, freqs, theta):
    x = extract_features(review, freqs)
    x[:, 1:] = (x[:, 1:] - mu) / sigma
    return sigmoid(np.dot(x, theta))
```

freqs

[Показати прихований результат](#)

X.shape

(1600, 5)

X[:5]

```
array([[ 1.          ,  0.18455983, -0.50079107,  0.24167889,  0.436649   ],
       [ 1.          ,  0.72891981, -0.66059697,  0.75142132,  0.83293551],
       [ 1.          , -0.24144569, -0.9135755 ,  0.27338899,  0.21291856],
       [ 1.          ,  0.98710001, -1.02078236,  0.98964739,  1.09025024],
       [ 1.          ,  0.14650681, -0.56844787,  0.29070268,  0.68885501]])
```

train_x[:5]

student elections . \ntracy flick (reese witherspoon) is an over-achiever with her hand raised at nearly every question

, way , way , high . \nmr . " m " (matthew broderick) , sick of the megalomaniac student , encourages paul , a popular-but-slow jock to run . \nand paul's nihilistic sister jumps in the race as well , for personal reasons . \nthe dark side of such sleeper success is that , because expectations were so low going in , the fact that this was quality stuff made the reviews even more enthusiastic than they have any right to be . \nyou can't help going in with the baggage of glowing reviews , which is in contrast to the negative baggage that the reviewers were likely to have . \n_election_ , a good film , does not live up to its hype . \nwhat makes _election_ so disappointing is that it contains significant plot details lifted directly from _rushmore_ , released a few months earlier . \nthe similarities are staggering : \ntracy flick (_election_) is the president of an extraordinary number of clubs , and is involved with the school play . \nmax fischer (_rushmore_) is the president of an extraordinary number of clubs , and is involved with the school play . \nthe most significant tension of _election_ is the potential relationship between a teacher and his student . \nthe most significant tension of _rushmore_ is the potential relationship between a teacher and his student . \ntracy flick is from a single parent home , which has contributed to her drive . \nmax fischer is from a single parent home , which has contributed to his drive . \nthe male bumbling adult in _election_ (matthew broderick) pursues an extramarital affair , gets caught , and his whole life is ruined . \nhe even gets a bee sting . \nthe male bumbling adult in _rushmore_ (bill murray) pursues an extramarital affair , gets caught , and his whole life is ruined . \nhe gets several bee stings . \nand so on . \nwhat happened ? \nhow is it that an individual screenplay (_rushmore_) and a novel (_election_) contain so many significant plot points , and yet both films were probably not even aware of each other , made from two different studios , from a genre (the high school geeks revenge movie) that hadn't been fully formed yet ? \neven so , the strengths of _election_ rely upon its fantastic performances from broderick , witherspoon , and newcomer jessica campbell , as paul's anti-social sister , tammy . \nbroderick here is playing the mr . rooney role from _ferris bueller_ , and he seems to be having the most fun he's had since then . \nwitherspoon is a revelation . \nit's early in the year , it's a comedy , and teenagers have little clout , but for my money , witherspoon deserves an oscar nomination . \nand once campbell's character gets going , like in her fantastic speech in the gymnasium , then you're won over . \none thing that's been bothering me since i've seen it . \nthere is an extraordinary amount of sexuality in this film . \ni suppose that , coming from mtv films , i should expect no less . . . \nbut the film starts off light and airy , like a sitcom . \nas the screws tighten , and the tensions mount , alexander payne decides to add elements that , frankly , distract from the story . \nit is bad enough that mr . m doesn't like tracy's determination to win at all costs , but did they have to throw in the student/teacher relationship ? \neven so , there's no logical reason why mr . m has an affair when he does . \nthere's a lot to like in _election_ , but the plot similarities to _rushmore_ , and the tonal nosedive it takes as it gets explicitly sex-driven , mark this as a disappointment . \n',

"you've got mail works alot better than it deserves to . \nin order to make the film a success , all they had to do was cast two extremely popular and attractive stars , have them share the screen for about two hours and then collect the profits . \nno real acting was involved and there is not an original or inventive bone in it's body (it's basically a complete re-shoot of the shop around the corner , only adding a few modern twists) . \nessentially , it goes against and defies all concepts of good contemporary filmmaking . \nit's overly sentimental and at times terribly mushy , not to mention very manipulative . \nbut oh , how enjoyable that manipulation is . \nbut there must be something other than the casting and manipulation that makes the movie work as well as it does , because i absolutely hated the previous ryan/hanks teaming , sleepless in seattle . \nit couldn't have been the directing , because both films were helmed by the same woman . \ni haven't quite yet figured out what i liked so much about you've got mail , but then again , is that really important ? \nif you like something so much , why even question it ? \nagain , the storyline is as cliched as they come . \ntom hanks plays joe fox , the insanely likeable owner of a discount book chain and meg ryan plays kathleen kelly , the even more insanely likeable proprietor of a family-run children's book shop called , in a nice homage , the shop around the corner . \nfox and kelly soon become bitter rivals because the new fox books store is opening up right across the block from the small business . \nlittle do they know , they are already in love with each other over the internet , only neither party knows the other person's true identity . \nthe rest of the story isn't important because

X[1595:]

```
array([[ 1.          , -1.18960992,  0.11844238, -0.73524531, -0.59646021],
       [ 1.          , -0.35092078,  0.02403228, -0.17795162, -0.18249859],
       [ 1.          , -0.80439215,  1.51574477, -1.05774694, -1.14891399],
       [ 1.          , -0.87526382,  0.05271532, -0.50344555, -0.66790053],
       [ 1.          , -1.05802458,  0.21661261, -0.67269262, -0.59742887]])
```

train_x[1595:]

operative transsexual , thank you very much . . . \na new-born woman for all purposes , with confusions and sensitivities that deserve treatment with a touch of delicacy . \npersonally , give me those cheap laughs any day . \ni'd rather enjoy obvious camp than endure the shallow exploration of a soft- hearted transsexual love story . \ndifferent for girls comes up with a promising enough proposition , but does nothing that surpasses the merely workmanlike with it . \nby refusing to make the audience really uncomfortable with the notion of kim and prentice's relation- ship , or at the very least feel uncomfortable for them (prentice for example , doesn't seem to give himself all that much grief when he becomes attracted to kim despite being a fight-picking , beer-drinking , macho kind of guy , and kim , for all her insecurities and fears is always saved any truly disturbing victimisation in the film) , and instead drawing them into a basically wimpy love story , the film manages to lobotomise itself . \nlightweight comedy as it's doomed to be , it could've still have hacked it with the requisitive endearing couple . \nbut this doesn't either , since even mrs doubtfire is sexier than steven mackintosh's kim foyle , and as either sex , he's just plainly a simpering wanker nobody should like . \nprentice does though , and that's never explained . \nbut then rupert graves gives us so heroic performance as prentice , the guy who never grew up (and if there is a bafta prize for most convincing portrayal of a rabid fan at a buzzcocks' concert , he should win that too) that i guess there's no reason to explain why he's so screwed up . \nif oddly enough you grow to like this show , you'll be wanting your happy ending , and the film won't fail you there either . \nside underling at kim's office gets her come-uppance . \nmisogynistic police officer who beats up prentice gets his come-uppance . \nthe couple prevail against a disagreeable society and come together in kim's apartment . \nkim's sister and her impotent sarge-type husband kiss and make up after a tiff (tv movie sub-plot with incidental juxtaposition) . \nand best of all , kim sheds her insecurities and learns how to ride the motorcycle and wear leathers , making prentice ride pillion . \nawwww . \nterrific closure . \nhappy ending . \nbut just what is it that's different for girls anyway ? \nif you figure that one out , let me know . \nthe flying inkpot rating system : * wait for the tv2 broadcast . \n * * a little creaky , but still better than staying at home with gotcha ! \n * * * pretty good , bring a friend . \n * * * * amazing , potent stuff . \n * * * * perfection . \nsee it twice . \n",

"a silly film that tries to be a black comedy but plays more like lightweight comedy , with its main asset being a beautiful film location along spain's mountainous coastline . \nhoward pigeon (jeff goldblum) is a 40-year-old mineral water salesman who is married for 12 years to a wife constantly seeing him . elizabeth (mini moore) , a former a-

```
Y = train_y.reshape(-1, 1)
```

```
Y.shape
```

```
(1600, 1)
```

Проведемо дослідження впливу швидкості навчання, кількості ітерацій та порогу класифікації на збіжність і точність моделі.

```
from matplotlib import pyplot as plt
```

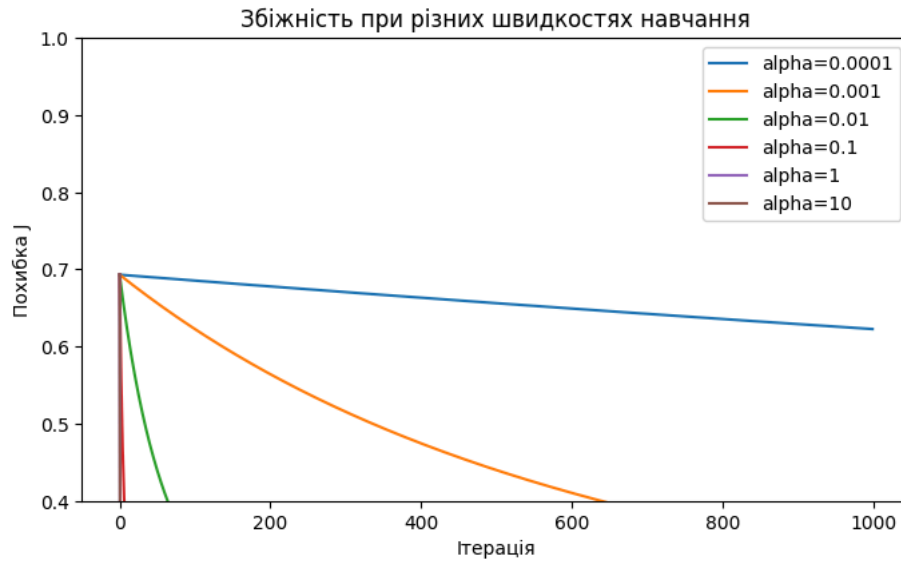
```
def acc(X_, y_, th, thr=0.5):
    return np.mean((sigmoid(X_ @ th) > thr).astype(int).ravel() == y_.ravel())
```

```
def cost(X_, y_, th):
    h = np.clip(sigmoid(X_ @ th), 1e-15, 1 - 1e-15)
    return float(-np.mean(y_ * np.log(h) + (1 - y_) * np.log(1 - h)))
```

```
rng = np.random.RandomState(42)
perm = rng.permutation(len(X))
cut = int(0.8 * len(X))
X_tr, X_val = X[perm[:cut]], X[perm[cut:]]
Y_tr, Y_val = Y[perm[:cut]], Y[perm[cut:]]
theta0 = np.zeros((X.shape[1], 1))
```

```
alphas = [1e-4, 1e-3, 1e-2, 1e-1, 1, 10]
plt.figure(figsize=(8, 4.5))
for a in alphas:
    J, th, hist = gradientDescent(X_tr, Y_tr, theta0, a, 1000)
    print(f"alpha={a}: J={J:.4f}, точність train={acc(X_tr, Y_tr, th):.4f}, val={acc(X_val, Y_val, th):.4f}")
    plt.plot(hist, label=f"alpha={a}")
plt.xlabel("Ітерація"); plt.ylabel("Похибка J")
plt.ylim(0.4, 1.0); plt.legend(); plt.title("Збіжність при різних швидкостях навчання")
plt.show()
```

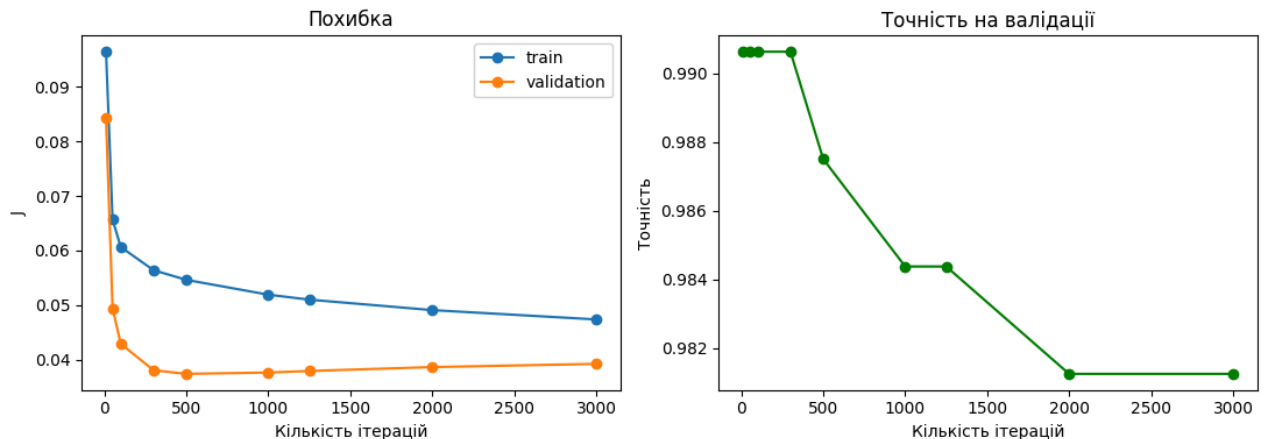
```
alpha=0.0001: J=0.6227, точність train=0.9773, val=0.9906
alpha=0.001: J=0.3265, точність train=0.9773, val=0.9906
alpha=0.01: J=0.1024, точність train=0.9797, val=0.9906
alpha=0.1: J=0.0607, точність train=0.9805, val=0.9906
alpha=1: J=0.0519, точність train=0.9836, val=0.9844
alpha=10: J=0.0416, точність train=0.9875, val=0.9812
```



```
best_alpha = 1.0
iters_list = [10, 50, 100, 300, 500, 1000, 1250, 2000, 3000]
tr_loss, val_loss, val_acc = [], [], []
for n in iters_list:
    _, th, _ = gradientDescent(X_tr, Y_tr, theta0, best_alpha, n)
    tr_loss.append(cost(X_tr, Y_tr, th))
    val_loss.append(cost(X_val, Y_val, th))
    val_acc.append(acc(X_val, Y_val, th))
    print(f"iters={n}: J_train={tr_loss[-1]:.4f}, J_val={val_loss[-1]:.4f}, val acc={val_acc[-1]:.4f}")

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(iters_list, tr_loss, marker="o", label="train")
ax[0].plot(iters_list, val_loss, marker="o", label="validation")
ax[0].set_xlabel("Кількість ітерацій"); ax[0].set_ylabel("J"); ax[0].legend(); ax[0].set_title("Помилка")
ax[1].plot(iters_list, val_acc, marker="o", color="green")
ax[1].set_xlabel("Кількість ітерацій"); ax[1].set_ylabel("Точність"); ax[1].set_title("Точність на валідації")
plt.tight_layout(); plt.show()
```

```
iters=10: J_train=0.0964, J_val=0.0842, val acc=0.9906
iters=50: J_train=0.0657, J_val=0.0494, val acc=0.9906
iters=100: J_train=0.0607, J_val=0.0429, val acc=0.9906
iters=300: J_train=0.0564, J_val=0.0381, val acc=0.9906
iters=500: J_train=0.0546, J_val=0.0374, val acc=0.9875
iters=1000: J_train=0.0519, J_val=0.0377, val acc=0.9844
iters=1250: J_train=0.0510, J_val=0.0380, val acc=0.9844
iters=2000: J_train=0.0491, J_val=0.0387, val acc=0.9812
iters=3000: J_train=0.0474, J_val=0.0393, val acc=0.9812
```



```
_, th, _ = gradientDescent(X_tr, Y_tr, theta0, best_alpha, 1000)
for thr in [0.3, 0.4, 0.5, 0.6, 0.65, 0.7]:
    print(f"nopir={thr}: val acc={acc(X_val, Y_val, th, thr):.4f}")
```

```
nopir=0.3: val acc=0.9844
nopir=0.4: val acc=0.9812
nopir=0.5: val acc=0.9844
nopir=0.6: val acc=0.9875
nopir=0.65: val acc=0.9875
nopir=0.7: val acc=0.9875
```

```
X_test = np.vstack([extract_features(r, freqs) for r in test_x])
X_test[:, 1:] = (X_test[:, 1:] - mu) / sigma
```

```
for a in [1e-4, 1e-3, 1e-2, 1e-1, 1, 10]:
    J_, th, hist = gradientDescent(X, Y, theta0, a, 1000)
    print(f"alpha={a}: J={J_:.4f}, train={acc(X, Y, th):.4f}, test={acc(X_test, test_y, th):.4f}")
```

```
alpha=0.0001: J=0.6220, train=0.9794, test=0.7925
alpha=0.001: J=0.3245, train=0.9800, test=0.7975
alpha=0.01: J=0.0998, train=0.9819, test=0.8000
alpha=0.1: J=0.0569, train=0.9825, test=0.7950
alpha=1: J=0.0495, train=0.9838, test=0.7925
alpha=10: J=0.0411, train=0.9856, test=0.7725
```

```
for n in [10, 50, 100, 300, 500, 1000, 1250, 2000, 3000]:
    _, th, _ = gradientDescent(X, Y, theta0, 0.1, n)
    print(f"iters={n}: J={cost(X, Y, th):.4f}, test={acc(X_test, test_y.reshape(-1,1), th):.4f}")
```

```
iters=10: J=0.3179, test=0.7925
iters=50: J=0.1360, test=0.7975
iters=100: J=0.0993, test=0.8000
iters=300: J=0.0695, test=0.7975
iters=500: J=0.0625, test=0.7975
iters=1000: J=0.0569, test=0.7950
iters=1250: J=0.0558, test=0.7975
iters=2000: J=0.0540, test=0.7975
iters=3000: J=0.0528, test=0.7975
```

Шляхом зміни швидкості навчання та кількості ітерацій було встановлено, що при малих α ($1e-4$, $1e-3$) модель недонавчена ($J = 0.62$ і 0.32), а при $\alpha = 10$ точність на тесті помітно падає (0.76). Найкращий компроміс між похибкою та точністю дає $\alpha = 0.1$ ($J = 0.056$, точність на тесті 0.80). Збільшення кількості ітерацій понад 1000 майже не зменшує похибку, а точність на тесті не зростає (0.7925 при 3000), тому обрано 1000 ітерацій.

```
J, theta, history = gradientDescent(X, Y, np.zeros((X.shape[1], 1)), 0.1, 1000)
print(f"Помилка після навчання: {J:.8f}.")
print(f"Вектор вагових коефіцієнтів: {[round(t, 8) for t in np.squeeze(theta)]}")
```

```
Помилка після навчання: 0.05694677.
Вектор вагових коефіцієнтів: [np.float64(0.08518801), np.float64(1.40655619), np.float64(-2.03654236), np.float64(1.54036877)
```

```
review = all_positive_reviews[0]
```

```
review
```

```
'films adapted from comic books have had plenty of success , whether they\'re about superheroes ( batman , superman , spawn ) , or geared toward kids ( casper ) or the arthouse crowd ( ghost world ) , but there\'s never really been a comic book li ke from hell before . \nfor starters , it was created by alan moore ( and eddie campbell ) , who brought the medium to a wh ole new level in the mid \'80s with a 12-part series called the watchmen . \nto say moore and campbell thoroughly researche d the subject of jack the ripper would be like saying michael jackson is starting to look a little odd . \nthe book ( or " graphic novel " if you will ) is over 500 pages long and includes nearly 30 more that consist of nothing but footnotes
```

```
predict_review(review, freqs, theta)
```

```
array([[0.92670076]])
```

```
if predict_review(review, freqs, theta) >= 0.5:
    print ('позитивний')
else:
    print ('негативний')
```

```
позитивний
```

```
def test_logistic_regression(test_x, test_y, freqs, theta):
    y_hat = []
    for review in test_x:
        y_pred = predict_review(review, freqs, theta)
        if y_pred > 0.5:
            y_hat.append(1)
        else:
            y_hat.append(0)
    accuracy = (np.sum(np.array(y_hat)==np.squeeze(test_y)))/len(test_x)
```

```
        return accuracy

tmp_accuracy = test_logistic_regression(test_x, test_y, freqs, theta)
print(f"Точність логістичної регресії: {tmp_accuracy:.4f}")
```

Точність логістичної регресії: 0.7950

```
test_mov="When you look at the first two Tom Holland-starrer Spider-Man movies, they had a breezy feel similar to the chara

p = predict_review(test_mov, freqs, theta)

if p > 0.65:
    print ('позитивний', p)
else:
    print ('негативний', p)
```

позитивний [[0.83408142]]

```
test_mov="These days, it seems like Colleen Hoover is at the epicenter of media. Whether it's the literary world or Hollywo

p = predict_review(test_mov, freqs, theta)

if p > 0.65:
    print ('позитивний', p)
else:
    print ('негативний', p)
```

негативний [[0.58814885]]

```
test_mov="""This saccharine, reverential biopic about controversial music legend Michael Jackson is set to be one of the wo

p = predict_review(test_mov, freqs, theta)

if p > 0.65:
    print ('позитивний', p)
else:
    print ('негативний', p)
```

негативний [[0.12493155]]